

EE2213 Introduction to Artificial Intelligence

Revision 1

Dr. Shaojing Fan
fanshaojing@nus.edu.sg

Q1. Which of the following statements about rational agents are correct? Select all that apply.

- a) Rational agents are assumed to know the complete environment dynamics and can predict how the environment will respond to their actions.
- b) A rational agent chooses actions that are expected to yield the highest performance according to a predefined measure.
- c) A rational agent that acts rationally will always achieve the true best outcome in every situation.
- d) Smart home devices, such as automatic lights, can be considered as rational agents.

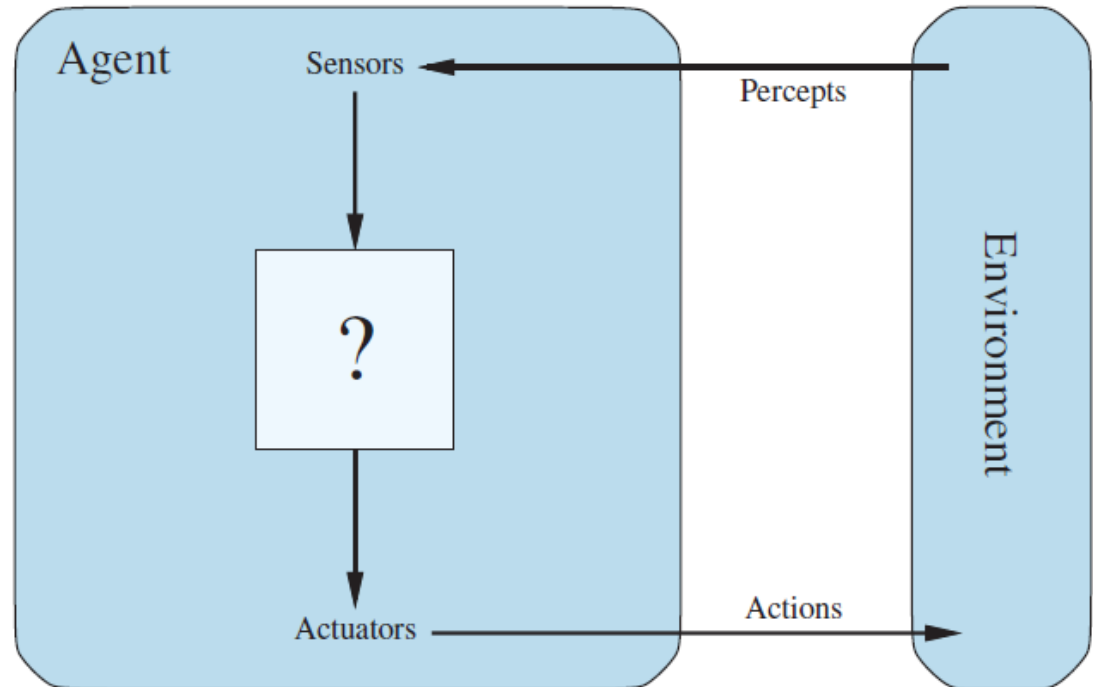
Recap: Definition of Agent

An **agent** is anything that **perceives** its environment through **sensors** and can **act** on its environment through **actuators**

A **percept** is the agent's **perceptual inputs** at any given instance

An **actuator** is the part of an agent that allows it to take **action** in the environment based on its decisions.

A **sensor** is a device or mechanism that allows an agent to perceive and gather information about its environment.

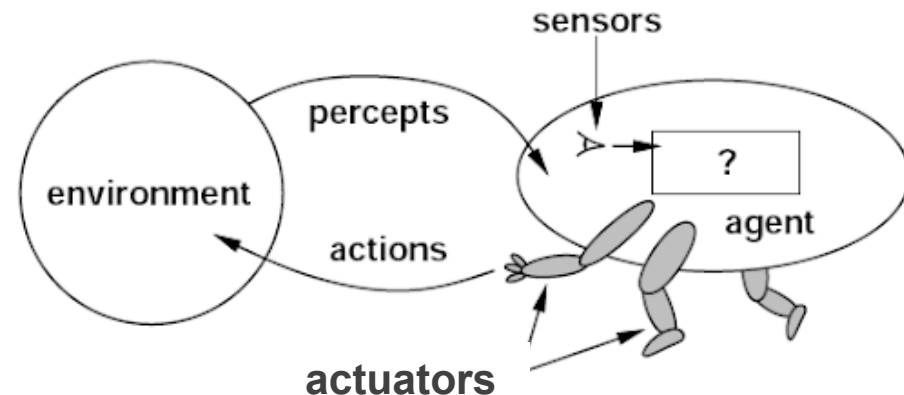


Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P54 – P57.

Recap: Definition of Rational Agent

Rational Agent:

- For each possible percept sequence P
- Select an action a **expected** to **maximize** the **performance measure**
- Base decision on:
 - *Evidence from the percept sequence*
 - *Built-in knowledge of the agent*



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P58.

Definition of Rational Agent (cont.)

- Are rational agents **omniscient**?

No – they only know what their sensors tell them.

- Are rational agents **clairvoyant**?

No – they may lack knowledge of the environment dynamics. They don't magically know the future or hidden parts of the environment.

- Do rational agents **explore** and **learn**?

Yes – in unknown environments these are essential

- Are rational agents **autonomous** (i.e., transcend initial program)?

Yes – as they learn, their behavior depends more on their own experience

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P58 – P60.

Q1. Which of the following statements about rational agents are correct? Select all that apply.

- a) Rational agents are assumed to know the complete environment dynamics and can predict how the environment will respond to their actions.
- b) A rational agent chooses actions that are expected to yield the highest performance according to a predefined measure. ✓
- c) A rational agent that acts rationally will always achieve the true best outcome in every situation.
- d) Smart home devices, such as automatic lights, can be considered as rational agents. ✓

Two levels of optimality:

- *Actual optimality*: what truly turns out to be best
- *Agent-optimality (rationality)*: what is best according to the agent's knowledge at the time

Q2. In AI, the PEAS framework helps describe an agent's task environment. Which of the following statements about PEAS components are true? Select all that apply.

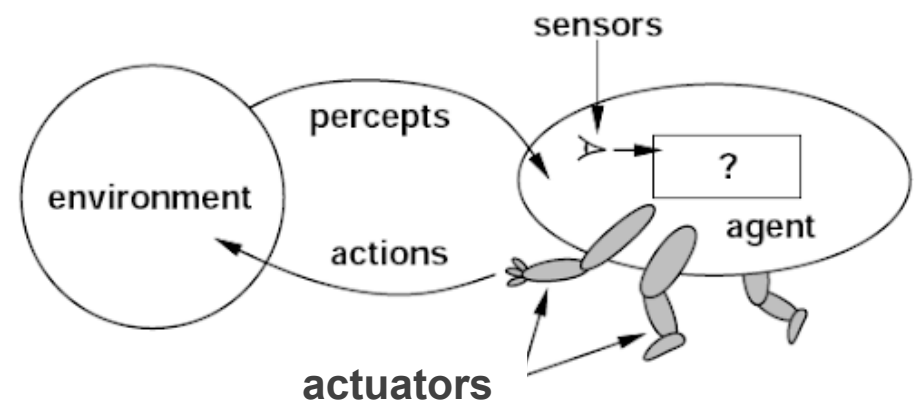
- (a) PEAS stands for Performance Measure, Environment, Actuators, and Sensors.
- (b) In an autonomous driving agent, steering, brakes, and accelerator are examples of actuators. The Environment includes external elements like roads, pedestrians, and other vehicles.
- (c) Sensors are used to perform physical actions such as turning or stopping.
- (d) The Environment includes everything the agent can interact with or be affected by.

Recap: Design a Rational Agent--Task Environment

To design a rational agent, we need to specify a **task environment** --- the setting/context in which the agent will operate and make decisions, and the problem specification the agent is meant to solve

PEAS: to specify a task environment

- **P**erformance measure
- **E**nvironment
- **A**ctuators
- **S**ensors



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P60 – P61.

PEAS: Example

Agent: autonomous vehicle

Performance measure (*evaluate how well it performs*)

- safety, speed, following traffic laws, comfort, maximize profits

Environment (*everything it interacts with*)

- roads, other traffic, pedestrians, customers

Actuators (*how it acts on the environment*)

- steering, accelerator, brake, signal, horn

Sensors (*where it gathers data*)

- cameras, LiDAR, speedometer, GPS



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P61.

Q2. In AI, the PEAS framework helps describe an agent's task environment. Which of the following statements about PEAS components are true? Select all that apply.

- (a) PEAS stands for Performance Measure, Environment, Actuators, and Sensors. ✓
- (b) In an autonomous driving agent, steering, brakes, and accelerator are examples of actuators. The Environment includes external elements like roads, pedestrians, and other vehicles. ✓
- (c) Sensors are used to perform physical actions such as turning or stopping.
- (d) The Environment includes everything the agent can interact with or be affected by. ✓

Q3. Which of the following statements about a search problem are true? Select all that apply.

- (a) The transition model defines the successor state resulting from applying an action to a current state.
- (b) The path cost depends only on the starting and ending states, not the sequence of actions taken.
- (c) The goal test determines whether a given state is a goal state.
- (d) Actions are the set of all possible states that the agent can occupy.

Recap: General search problems formulation

In general, a search problem is defined by:

1. **State space:** a set S containing all possible states of the environment.
2. **Initial state:** $s_i \in S$, indicating where the agent starts.
3. **Actions:** a set A containing possible moves from a state.
 $\forall s \in S: A = \text{ACTIONS}(s) = \text{actions executable in } s$
4. **Transition model :** Result of applying an action.
 $\forall a \in \text{ACTIONS}(s), \text{RESULT}(s, a) \rightarrow s', s' \text{ is called a successor of } s$
5. **Action cost:** Cost of applying an action $c(s, a, s')$
6. **Goal state(s):** Desired state(s) with a desired property
The agent uses a **goal test** to check if the current state is a goal state
 $\text{IS-GOAL}(s)$: s is a goal state if $\text{IS-GOAL}(s)$ is true

Q3. Which of the following statements about a search problem are true? Select all that apply.

- (a) The transition model defines the successor state resulting from applying an action to a current state. ✓
- (b) The path cost depends only on the starting and ending states, not the sequence of actions taken.
- (c) The goal test determines whether a given state is a goal state. ✓
- (d) Actions are the set of all possible states that the agent can occupy.

Q4. What is the major disadvantage of graph search as opposed to tree search?

- (a) Visual representations of graph search are less appealing but graph search is otherwise the same as tree search
- (b) Graph search is unable to detect repeated states
- (c) There is no disadvantage since graph search is just another name for tree search
- (d) Graph search can require a lot of memory because it keeps track of all possible states

Recap: Two fundamental search strategies

Tree Search

- Explores the state space as a tree
- Treats each occurrence of a state via a different path as a distinct node
- May generate redundant states

Graph Search

- Explores the state space as a graph
- Tracks visited states to avoid revisiting
- Prevents redundancy and loops

Reference: <https://www.youtube.com/watch?v=Y1VywhuRFIs>

Recap: Graph search vs Tree search

Tree search

```
function TREE-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    else expand the node, and add the resulting child nodes to the frontier
  end
```

Graph search

```
function GRAPH-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  Initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    add the node to the explored set
    expand the node, and add the resulting nodes to the frontier
    only if not in the frontier or explored set
  end
```

Reference: <https://www.youtube.com/watch?v=Y1VywhuRFIs>

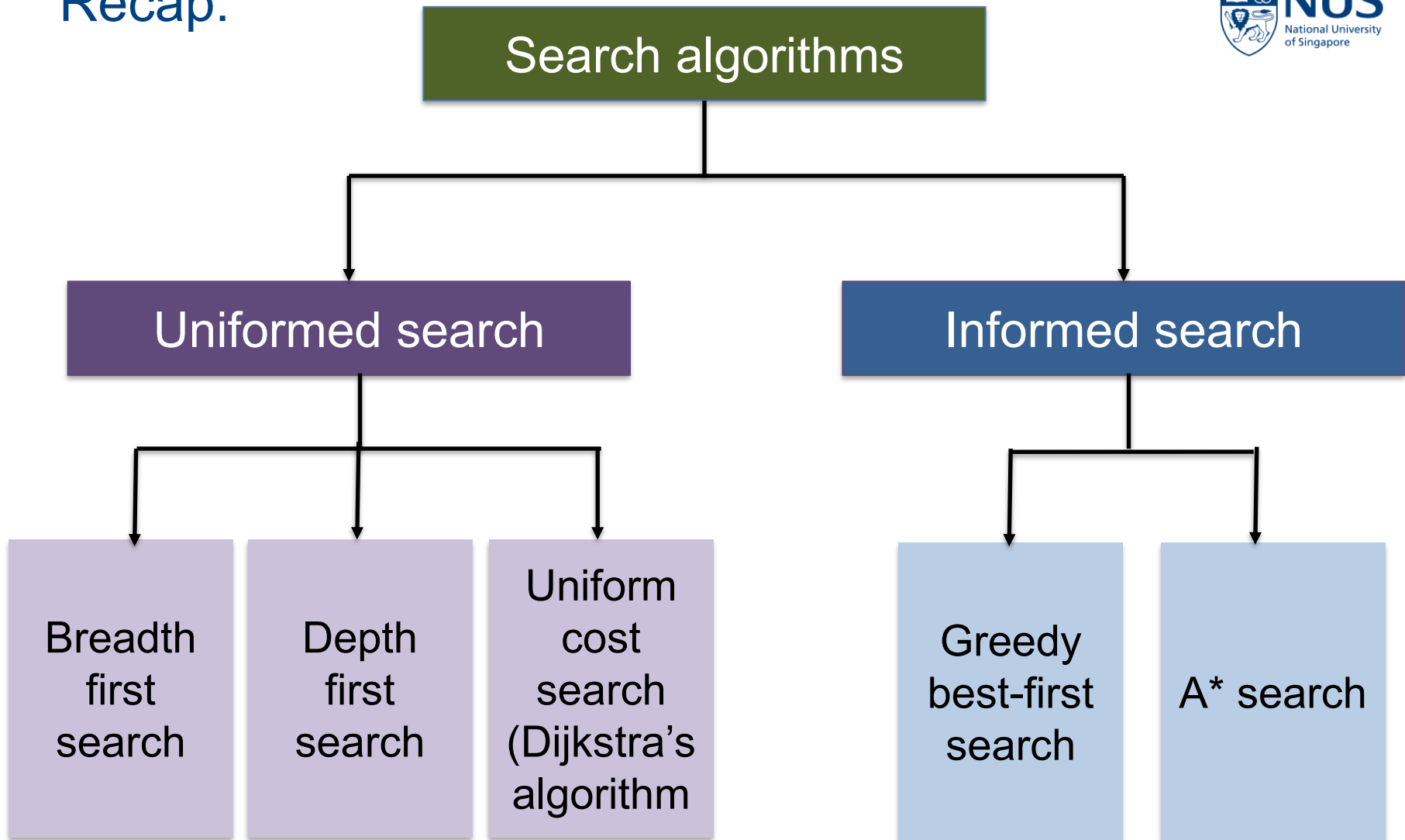
Q4. What is the major disadvantage of graph search as opposed to tree search?

- (a) Visual representations of graph search are less appealing but graph search is otherwise the same as tree search
- (b) Graph search is unable to detect repeated states
- (c) There is no disadvantage since graph search is just another name for tree search
- (d) Graph search can require a lot of memory because it keeps track of all possible states ✓

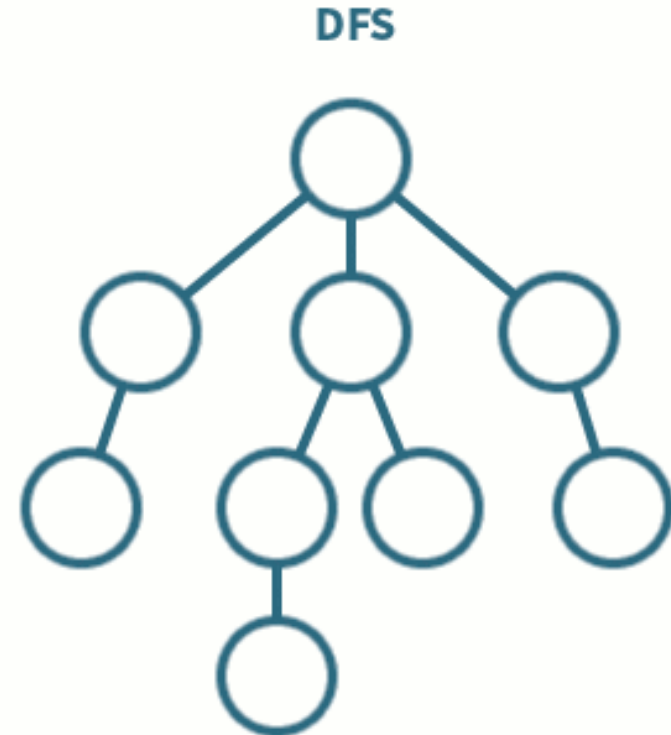
Q5. What happens when A^* search is used with a heuristic function $h(n) = 0$ for all nodes?

- (a) It behaves like Breadth-First Search, ignoring edge costs entirely.
- (b) It behaves randomly, since the heuristic provides no guidance.
- (c) It behaves like Dijkstra's algorithm, expanding nodes based on the lowest total path from the start node to that node ($g(n)$).
- (d) It behaves like Greedy Best-First Search, favoring nodes that appear closer to the goal.

Recap:

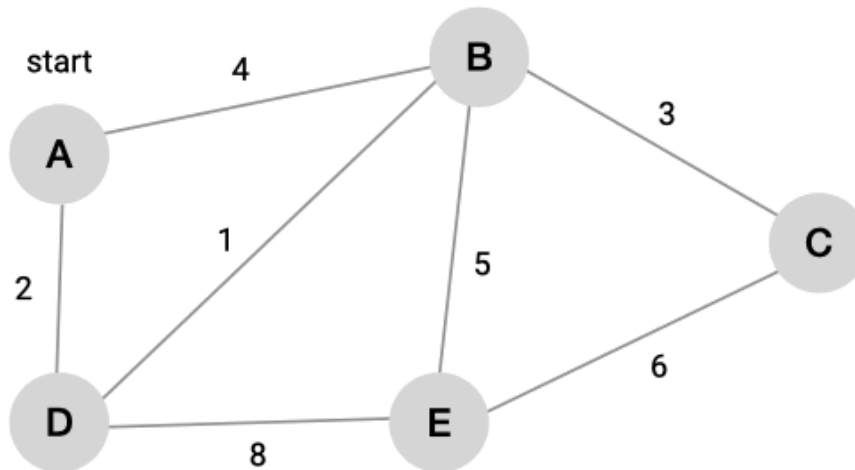


Recap: BFS vs DFS



Animation source: <https://medium.com/analytics-vidhya/a-quick-explanation-of-dfs-bfs-depth-first-search-breadth-first-search-b9ef4caf952c>

Recap: Dijkstra's algorithm



Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

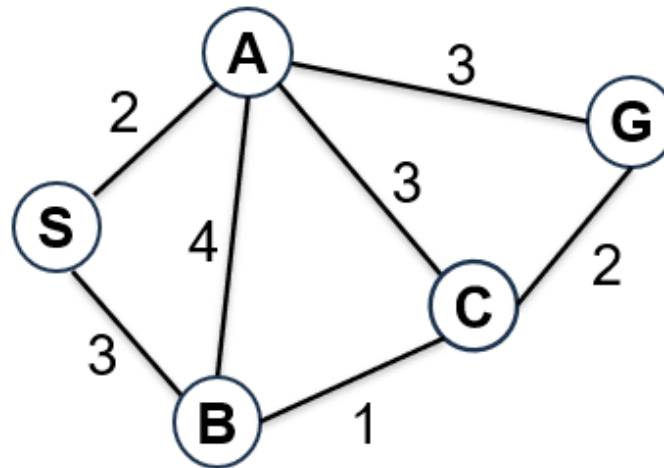
Animation source: <https://memgraph.com/docs/advanced-algorithms/deep-path-traversal>

Recap: BFS vs DFS vs Dijkstra's algorithm

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)	Dijkstra's (Uniform-cost search)
Search style	Explores all nearby nodes first (level by level)	Goes as deep as possible before backtracking	Expands the node with the <i>lowest total cost</i> from the source (initial state)
Data structure used	Queue (FIFO - First in, first out)	Stack (LIFO - Last in, first out)	Priority queue (based on path cost)
Will it always find a solution if one exists? (Completeness)	Yes, if a solution exists	Not guaranteed – may get stuck in deep or infinite paths	Yes
Will it find the shortest solution? (Optimality)	Yes, if all steps cost the same	No, may find a longer or less efficient path	Yes, even when edge costs differ
Can it handle different edge costs?	No – assumes all edges have equal cost	No – ignores cost	Yes – works with any non-negative edge cost
Time efficiency	Slower if the solution is deep	Can be faster if the solution is deep	Slower in large graphs with varying costs
Memory usage	High – remembers all paths at one level	Low – tracks only one path at a time	High – must track cumulative costs for all paths
Best for	Finding shortest paths (e.g., in maps) Small search space	Exploring puzzles or scenarios with deep solutions Large or deep search spaces	Finding the cheapest/least-cost path in weighted graphs (e.g., shortest time, lowest expense)
When to use	Need shortest route Equal step cost Enough memory	Memory is limited When solution may be far/deep Don't need shortest path	Need shortest route with different edge costs; can afford higher memory usage

Recap: Greedy best-first search

Only considers heuristics $h(n)$



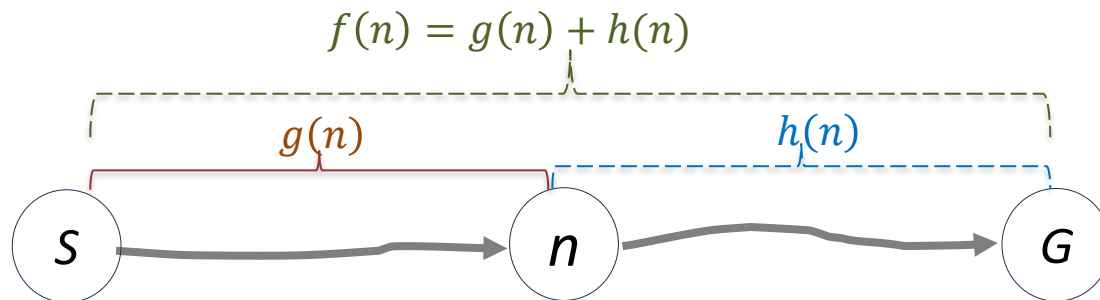
node	$h(n)$
S	6
A	4
B	3
C	1
G	0

$S \rightarrow B \rightarrow C \rightarrow G$

Recap: A*--Where *heuristics* meet *reality*

It evaluates nodes by combining $g(n)$, the actual cost to reach the node, and $h(n)$, the estimated cost to get from the node to the goal.

$$f(n) = g(n) + h(n) \quad \leftarrow \quad f(n): \text{Estimated total cost of the cheapest path from the start to the goal via node } n$$

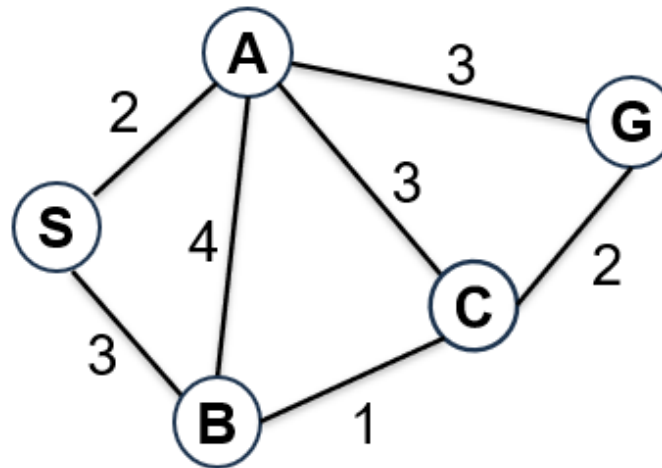


$g(n)$: Actual cost of the path from the start node to the current node n . (for start node: $g(n) = 0$)

$h(n)$: Estimated cost of the cheapest path from the current node n to the goal. (for goal node: $h(n) = 0$)

Recap: A* search

Considers both heuristics $h(n)$ and actual cost $g(n)$



node	$h(n)$
S	6
A	4
B	3
C	1
G	0

$S \rightarrow A \rightarrow G$

Recap: Overall summary

Feature	Uninformed search	Informed search
Search Algorithms	BFS, DFS, UCS (Dijkstra's algorithm)	Greedy best-first search, A* search
Additional knowledge available?	No (explores blindly, only uses problem definition, i.e., start, actions, goal)	Yes (uses domain-specific knowledge/heuristics to guide search)
Complete?	BFS & UCS (Dijkstra's algorithm): Yes DFS: No (can get stuck in infinite paths)	Greedy best-first: No (can get stuck in local optima) A*: Generally yes (finite search space and non-negative edge cost)
Optimal?	BFS: Yes (if all step costs are equal/unweighted graph) DFS: No (finds any path, not necessarily shortest) UCS (Dijkstra's algorithm): Yes (finds optimal path in weighted graphs with non-negative costs)	Greedy best-first: No (may find suboptimal path instead) A*: Yes (with admissible & consistent heuristic)
Efficiency	Generally less efficient for large search spaces, as it explores broadly/deeply without guidance. May explore many irrelevant paths	Generally more efficient for large search spaces, as heuristics guide it towards the goal.
When to use	Small or simple problems No heuristic available	Larger or more complex problems When good heuristic is available

Q5. What happens when A^* search is used with a heuristic function $h(n) = 0$ for all nodes?

- (a) It behaves like Breadth-First Search, ignoring edge costs entirely.
- (b) It behaves randomly, since the heuristic provides no guidance.
- (c) It behaves like Dijkstra's algorithm, expanding nodes based on the lowest total path from the start node to that node ($g(n)$). ✓
- (d) It behaves like Greedy Best-First Search, favoring nodes that appear closer to the goal.

Q6. Which of the following statements about admissible and consistent heuristics is true?
Assume all heuristics are non-negative.

- (a) Every heuristic that is admissible is also consistent.
- (b) A consistent heuristic may overestimate the true cost to the goal.
- (c) Every consistent heuristic is also admissible.
- (d) Admissibility is a stricter condition than consistency.

Recap: Admissible heuristics

A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true cost/actual cost** to reach the goal state from n .

Admissible heuristics are **optimistic**: they **never overestimate** the cost to reach the goal, always estimating it as **less than or equal** to the actual cost.

Example:

Driving from City A to B

- Actual road distance: 100 km
- Straight-line (Euclidean) distance as heuristic: 80 km

The heuristic **never overestimates** → **Admissible**



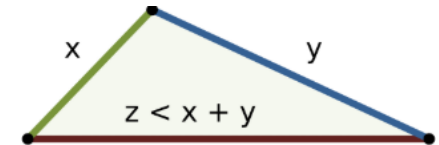
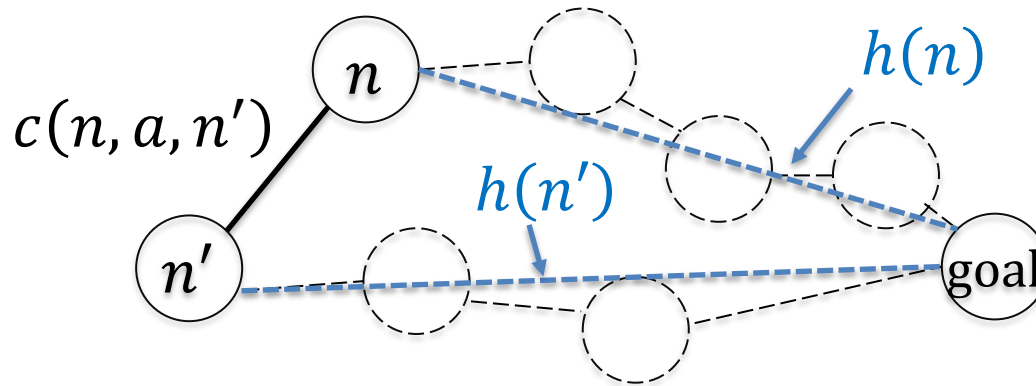
Recap: Consistent heuristics

A heuristic $h(n)$ is **consistent** (or monotonic) if for every node n , every successor n' of n generated by any action a :

$$h(n) \leq c(n, a, n') + h(n')$$

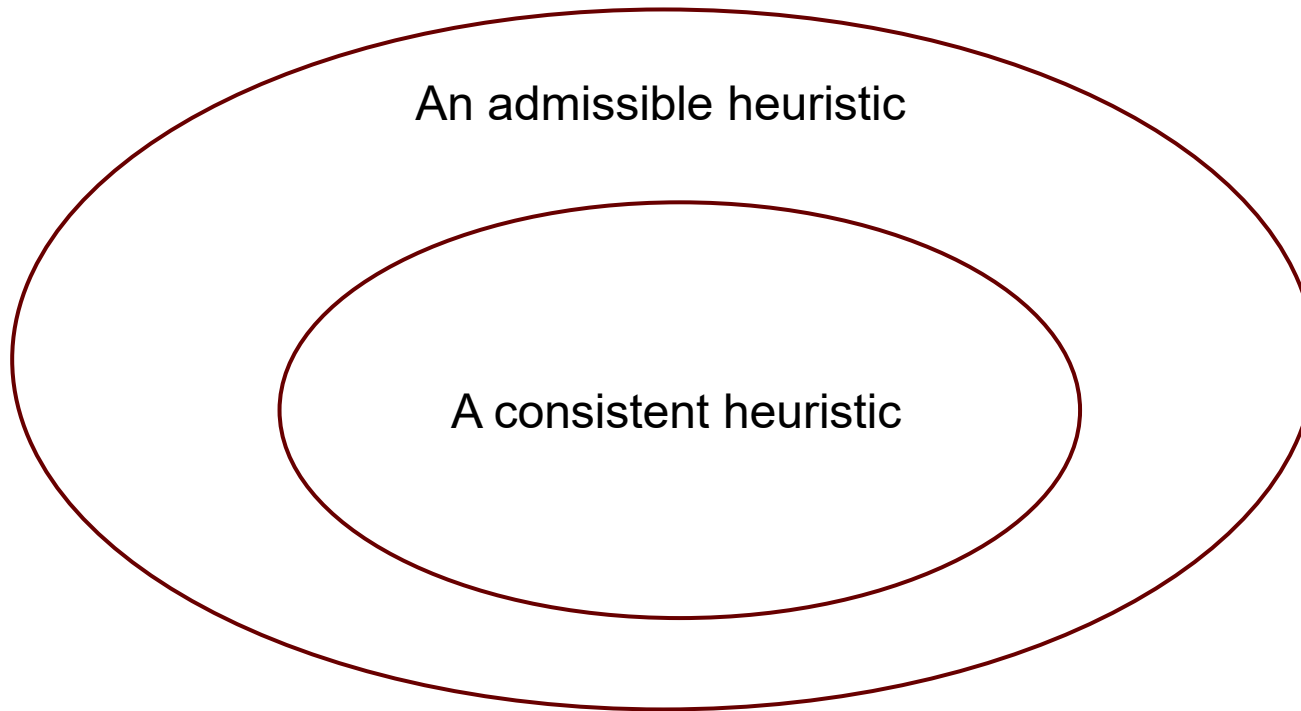
$c(n, a, n')$ is the actual cost of applying action a to go from state n to n'

Consistency is a version of triangle inequality.



Consistency is a **stricter requirement** than admissibility.

Recap: Admissible and Consistent heuristics



A consistent heuristic is always admissible.

However, an admissible heuristic is not always consistent.

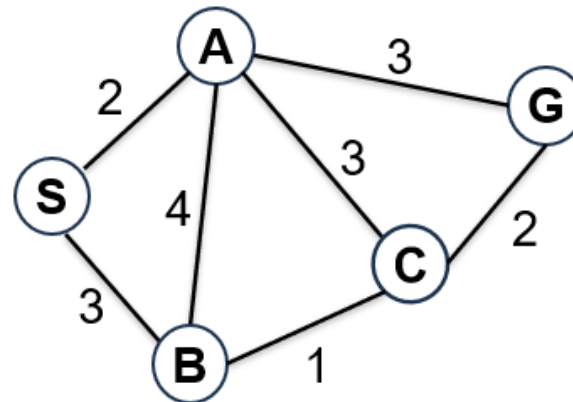
Q6. Which of the following statements about admissible and consistent heuristics is true?
Assume all heuristics are non-negative.

- (a) Every heuristic that is admissible is also consistent.
- (b) A consistent heuristic may overestimate the true cost to the goal.
- (c) Every consistent heuristic is also admissible. ✓
- (d) Admissibility is a stricter condition than consistency.

Q7. Please perform greedy best-first search on the graph below, using a graph search strategy (i.e., nodes are not revisited once expanded). The numbers on the edges represent the step cost between nodes, and the heuristic values (estimated cost to the goal G) for each node are shown in the table on the right side of the graph.

Starting from node S, which of the following is a possible total cost of the path found by greedy best-first search to reach the goal node G?

- (a) 5
- (b) 6
- (c) 7
- (d) 8
- (e) 9

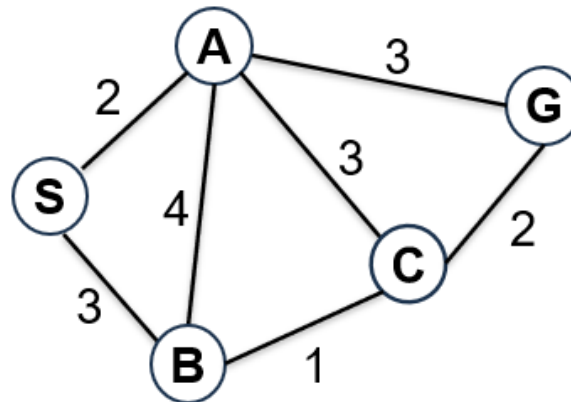


node	$h(n)$
S	6
A	4
B	3
C	1
G	0

Q7. Please perform greedy best-first search on the graph below, using a graph search strategy (i.e., nodes are not revisited once expanded). The numbers on the edges represent the step cost between nodes, and the heuristic values (estimated cost to the goal G) for each node are shown in the table on the right side of the graph.

Starting from node S, which of the following is a possible total cost of the path found by greedy best-first search to reach the goal node G?

- (a) 5
- (b) 6 ✓
- (c) 7
- (d) 8
- (e) 9



node	$h(n)$
S	6
A	4
B	3
C	1
G	0

